BACKEND ENGINEERING · HLD INTERVIEW PREP

Consistent Hashing

Complete Study Notes

Hash Ring · Virtual Nodes · Node Add/Remove
Load Balancing · Distributed Cache · DB Sharding

TOPICS COVERED

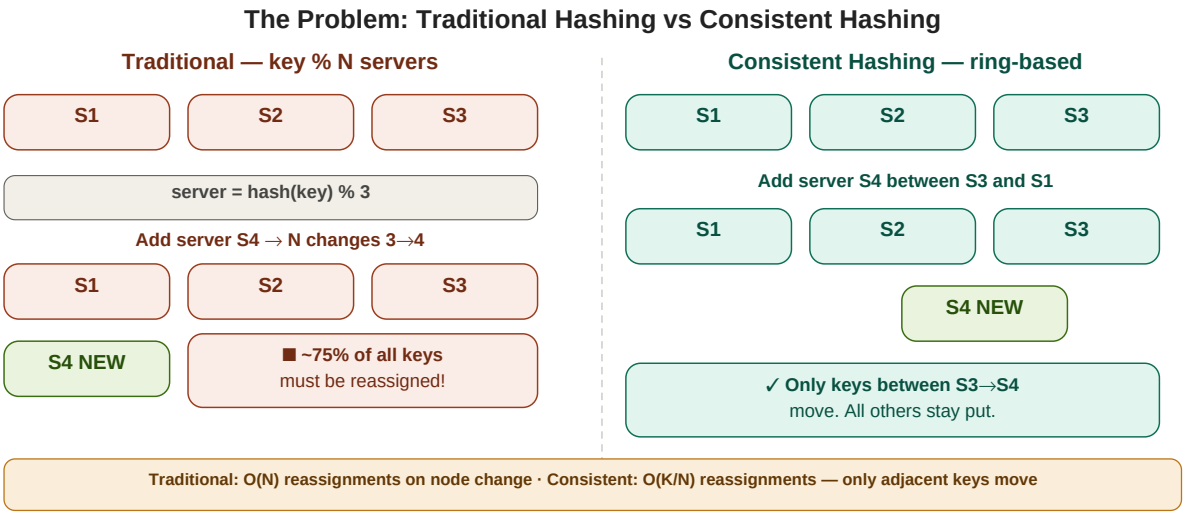
Table of Contents

1. What is Hashing?

A **hash function** converts an input (username, URL, key) into a fixed-length numeric value. This value is used to decide which server or bucket stores or processes that data. Traditional modular hashing uses: **server = hash(key) % N** where N is the number of servers.

2. Problem with Traditional Hashing

When N changes (a server is added or removed), the formula **hash(key) % N** produces different results for almost all keys. This forces a massive, expensive redistribution of data across servers.



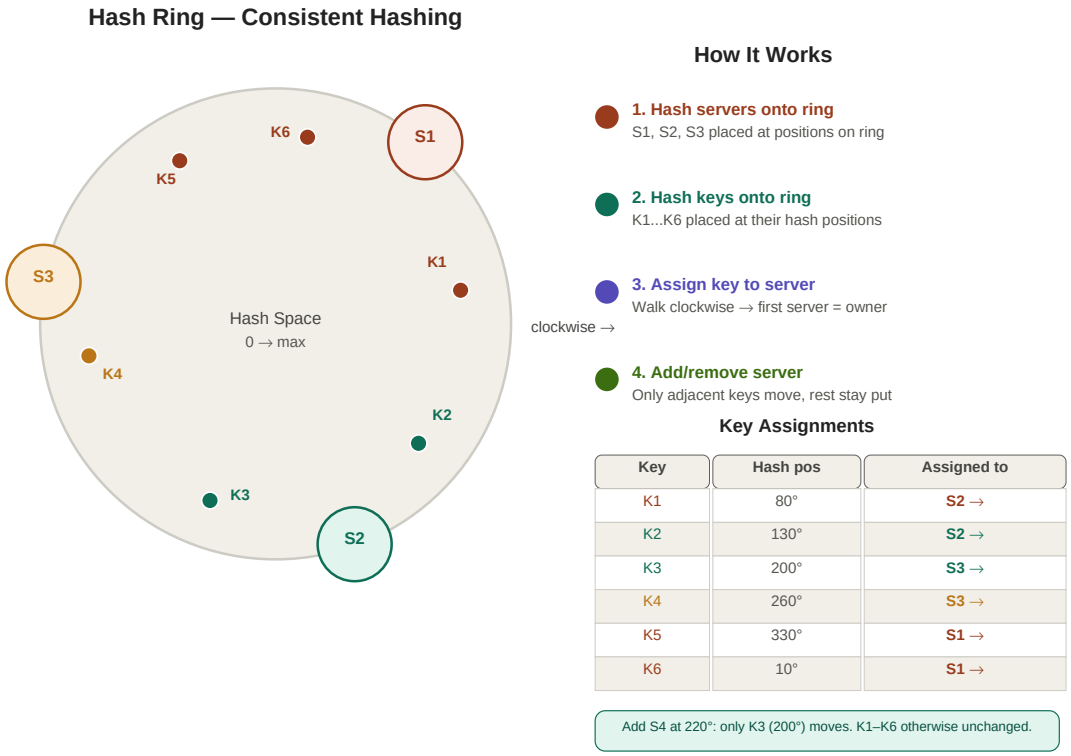
Traditional hashing: ~75% keys move on any node change · Consistent hashing: only adjacent keys move

Example: 6 servers, hash("user_123") % 6 = 3 → Server 3. Add a 7th server: hash("user_123") % 7 = 2 → Server 2. Data must move!

At scale: 1M keys, add 1 server → ~857,000 keys need to be relocated. This causes cache misses, DB overload, and service disruption.

3 & 4. Consistent Hashing — The Hash Ring

Consistent hashing solves the redistribution problem by placing both **servers and keys** on the same circular hash space (the ring). A key is always assigned to the **first server clockwise** from its position. When a server is added or removed, only the keys in the adjacent arc need to move.



Hash ring: servers at fixed positions, keys assigned to next clockwise server · only adjacent segment affected on change

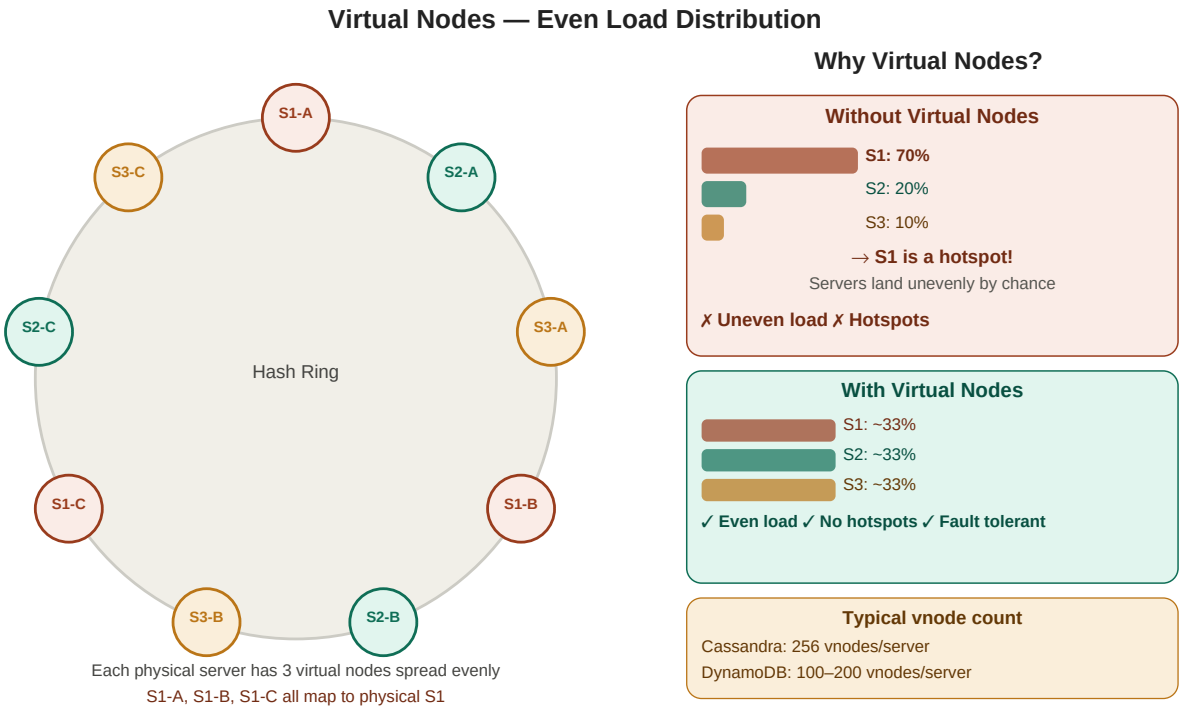
Step-by-step algorithm

- **Hash servers** onto ring: server_pos = hash(server_id) % ring_size
- **Hash keys** onto ring: key_pos = hash(key) % ring_size
- **Assign key**: walk clockwise from key_pos — first server encountered is the owner
- **Add server S4**: insert at its hash position; only keys between prev_server and S4 move
- **Remove server**: its keys reassign to the next clockwise server

Property	Traditional hash(key) % N	Consistent Hashing
Key mapping	Fixed modulo formula	Clockwise on ring
Node addition	~N-1/N keys must move (massive)	Only K/N keys move (minimal)
Node removal	~N-1/N keys must move	Only adjacent keys move
Hotspot risk	Possible	Mitigated via virtual nodes
Complexity	O(1) lookup	O(log N) with sorted ring
Scalability	Poor — downtime on node change	Excellent — elastic scaling

5. Virtual Nodes (vnodes)

With basic consistent hashing, servers may occupy unequal arcs on the ring — one server might own 60% of the ring while another owns 10%. **Virtual nodes** solve this by placing each physical server at multiple positions on the ring, ensuring even load distribution.



Virtual nodes: each server gets 3+ positions spread evenly — S1-A, S1-B, S1-C all map to physical S1

How virtual nodes work

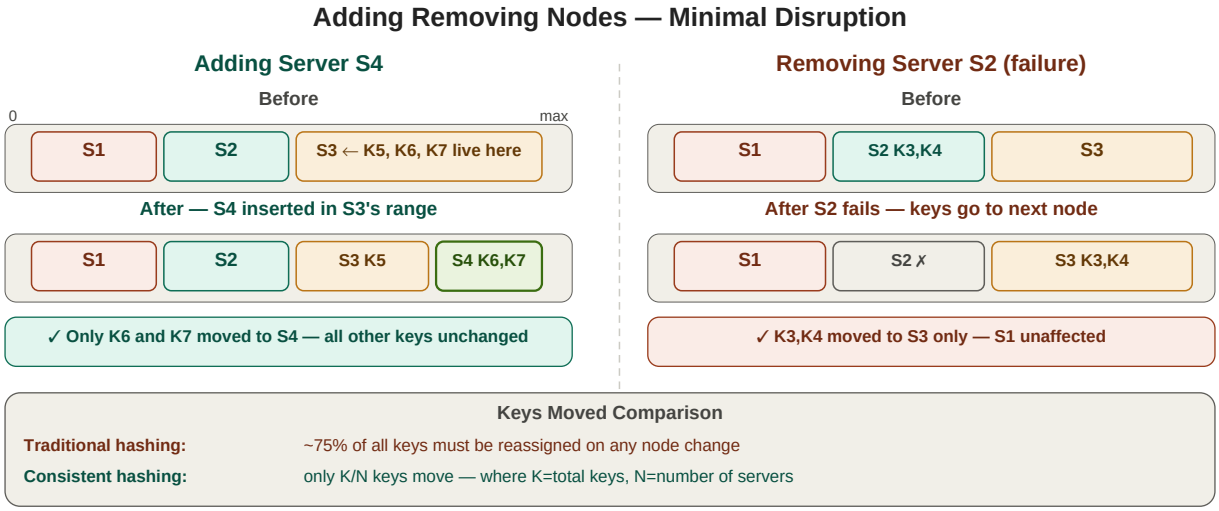
- Physical server S1 gets 3 positions: S1-A, S1-B, S1-C — each hashed to a different ring position
- These positions are spread evenly so each server owns ~1/N of the ring regardless of hash collisions
- A key hitting S1-B is served by physical server S1
- More vnodes = smoother distribution but more memory for the ring lookup structure

Cassandra: 256 virtual nodes per physical server (configurable).

DynamoDB: 100–200 virtual nodes per server.

Rule of thumb: more heterogeneous hardware → more vnodes on stronger servers to proportionally increase their load share.

6. Adding & Removing Nodes



Adding S4: only keys in S3→S4 arc move · Removing S2: its keys reassign to S3 only · all others untouched

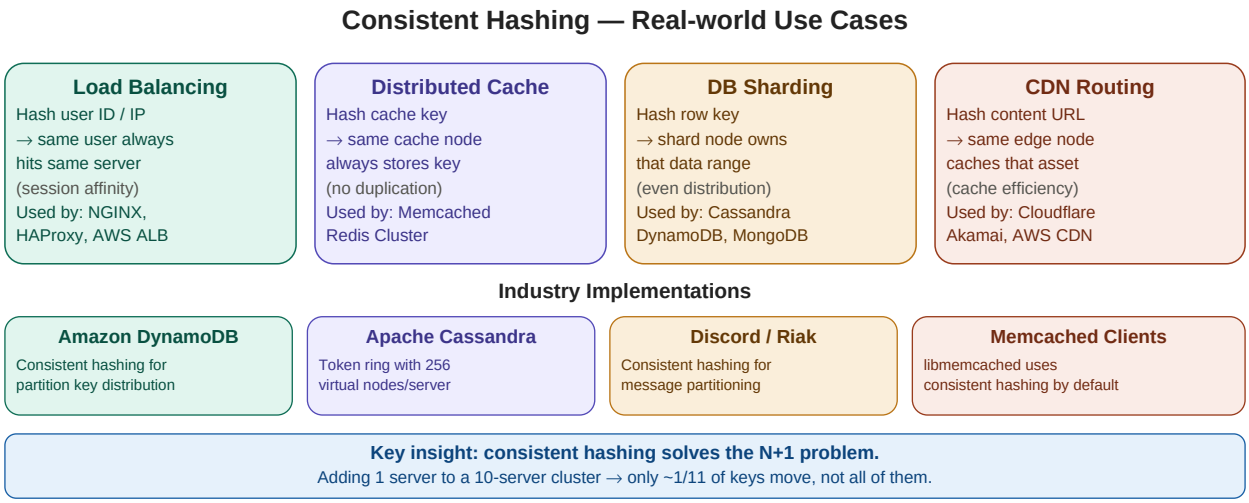
Adding a node

- Compute hash position of new server and insert into ring
- Identify the arc from previous server to new server
- Move only the keys in that arc to the new server — everything else is undisturbed
- In a 10-server cluster, adding 1 server moves ~1/11 ≈ 9% of keys

Removing a node (failure)

- Server is removed from the ring
- Keys that were on that server are reassigned to the next clockwise server
- Only those keys move — all other servers keep their keys intact
- No full reshuffle, no cache invalidation cascade

7. Real-world Use Cases & Industry



Consistent hashing powers load balancers, distributed caches, database sharding, and CDN routing

8. Advantages & Limitations

Advantages	Limitations
Minimal data movement on node changes — only K/N keys	Higher implementation complexity than modulo hashing
Horizontal scaling without downtime	Ring lookup is O(log N) vs O(1) for modulo
Fault tolerance — node failure affects only adjacent keys	Without vnodes, uneven arc sizes cause hotspots
Works with heterogeneous hardware via weighted vnodes	Very high churn (rapid add/remove) still costly
Enables elastic auto-scaling in cloud environments	Consistent hash functions must be chosen carefully
Session affinity — same user → same server	Cross-shard operations still require coordination

9. Quick Revision & Interview Questions

Quick Revision

→ Traditional hashing: server = hash(key) % N. Adding/removing N causes ~75% of keys to move.
→ Consistent hashing places both servers AND keys on a circular hash ring.
→ Key assignment: walk clockwise from key position — first server encountered is the owner.
→ Adding a server: only keys in the arc between prev_server and new_server move.
→ Removing a server: only its keys move to the next clockwise server.
→ Without vnodes: unequal arc sizes → one server gets disproportionate load (hotspot).
→ Virtual nodes: each physical server gets multiple ring positions → even load distribution.
→ Cassandra uses 256 vnodes/server. DynamoDB uses 100–200 vnodes/server.
→ Key formula: adding 1 server to N-server cluster moves ~1/(N+1) of all keys.
→ Used in: load balancers (session affinity), Redis/Memcached clusters, Cassandra, DynamoDB, CDNs.
→ Ring lookup complexity: O(log N) using a sorted array with binary search.
→ Hotspot still possible if hash function distributes poorly — good hash function (MurmurHash) required.

Interview Questions

- 1. What is the problem with traditional modular hashing when servers are added or removed?
- 2. Explain consistent hashing. How does the hash ring work?
- 3. How are keys assigned to servers in consistent hashing?
- 4. What happens when a new server is added to a consistent hashing ring?
- 5. What happens when a server fails in consistent hashing?
- 6. What are virtual nodes? Why are they needed?
- 7. How many keys move when you add 1 server to a cluster of N servers?
- 8. How does consistent hashing help with load balancing?
- 9. What is the time complexity of a key lookup in consistent hashing?
- 10. Name three real-world systems that use consistent hashing.
- 11. What are the limitations of consistent hashing?
- 12. How does Cassandra use consistent hashing?

Key Terms

Hash Function	Converts input to fixed-size numeric value — determines ring/bucket position
Hash Ring	Circular space [0, max] where both servers and keys are placed
Consistent Hashing	Ring-based scheme minimising key movement on node add/remove
Key Assignment	Walk clockwise from key position — first server is the owner
Virtual Node (vnode)	Multiple logical ring positions for one physical server — ensures even distribution
Rebalancing	Redistributing keys after node addition or removal — minimal in consistent hashing
Hotspot	One server receiving disproportionate load due to unequal arc sizes
K/N keys	Number of keys that move when adding 1 server to N-server cluster

MurmurHash	Fast, well-distributing hash function commonly used for consistent hashing rings
Session Affinity	Same client IP always maps to same server — achieved via IP hash on ring
Modulo Hashing	Traditional server = hash(key) % N — breaks on node count change